

最优可变长数组

Optimal Resizable Arrays

Robert E. Tarjan · Uri Zwick

arXiv:2211.11009v2 (2023-05-29) 中文翻译

说明：本译稿保留原文的章节编号、定理编号、公式与伪代码结构。术语约定：resizable array 译为“可变长数组”，grow/shrink 译为“增长/收缩”，block 译为“块”，worst-case time 译为“最坏情形时间”，amortized time 译为“摊还时间”。

摘要

可变长数组是一种能够通过末端或两端添加、删除元素而增长和收缩的数组，同时仍支持根据下标在常数时间内访问其中每个元素。由于数组大小（即其中元素的数量）会随时间变化，高空间效率地维护可变长数组需要动态内存管理。标准的倍增技术可用 $O(N)$ 空间维护大小为 N 的数组，每次操作的摊还时间为 $O(1)$ ，甚至最坏情形时间也可做到 $O(1)$ 。Sitarski 以及 Brodnik 等人给出了更好的方案：只用 $N+O(\sqrt{N})$ 空间维护大小为 N 的可变长数组，同时每次操作仍为 $O(1)$ 时间。Brodnik 等人还给出了一个简单证明，说明这一界是最优的。

本文区分两类空间：一类是存储可变长数组并访问其元素所需的常驻空间；另一类是数组增长或收缩时可能短暂需要的临时空间。对任意整数 $r \geq 2$ ，我们证明：若增长和收缩期间可短暂使用 $N+O(N^{1-1/r})$ 空间，则只用 $N+O(N^{1/r})$ 空间即可存储并访问大小为 N 的数组。按下标访问元素的最坏情形时间为 $O(1)$ ，增长和收缩操作的摊还时间为 $O(r)$ 。通过对一个“增长游戏”的精确分析，我们证明：对于一个广泛的数据结构类别，只要存储数组时仅使用 $N+O(N^{1/r})$ 空间，增长操作的摊还代价就是 $\Omega(r)$ ，即使仅允许增长和访问操作也是如此。除 $r=2$ 外，增长和收缩操作的时间无法变为最坏情形界。

1 引言

数组和可变长数组可能是使用最广泛的数据结构。令人意外的是，关于它们似乎仍有新的内容。本文给出若干简单的新型可变长数组实现；在许多情形下，它们比此前所有方案都更节省内存。我们的视角以理论为主，但这些实现也可能具有实际意义。

大小为 N 的数组是一种保存序列 $a_0, a_1, \dots, a_{N-1}$ 的数据结构。对任意 $0 \leq i < N$ ，都能在常数时间内读取或修改第 i 个元素。数组通常实现为一段连续的 N 个内存字，每个字保存一个元素或一个元素指针；借助机器的随机访问能力即可在常数时间内读写。按此定义，数组本质上具有固定大小。

可变长数组也称动态数组或动态表，它的大小可以增减，同时仍能在常数时间内读取或修改任意第 i 个元素。减小数组大小很容易：直接停止使用已分配空间的一部分即可，但这会浪费内存。增大数组更困难，因为当前连续内存块之后的内存字可能已被其他用途占用。因此，维护可变长数组需要动态内存管理。

为简化讨论，本文大部分内容假定数组只在“远端”增长和收缩：大小从 N 增到 $N+1$ 时加入新元素 a_N ；从 N 减到 $N-1$ 时元素 a_{N-1} 消失。这类可变长数组推广了栈。所有结果都可扩展到两端均可增长、收缩的数组，从而推广双端队列；只需用两个单端可变长数组即可实现。

教科书中的标准方案是倍增（或几何扩张/收缩）技术。先分配一个初始数组；数组满时，分配一个例如两倍大小的新数组，把所有元素复制过去；数组使用率低于某个阈值（如四分之一）时，再分配一个例如一半大小的新数组并复制。旧数组随后释放。简单的摊还分析表明，每次增减大小的摊还代价为 $O(1)$ ，元素访问和修改的最坏情形时间仍为 $O(1)$ 。利用后台重建技术，增减操作也可去摊还化为 $O(1)$ 最坏情形时间。

据文献 [16]，增长因子在 1.25 到 2 之间的几何扩张/收缩技术，被用于 Java 的 `ArrayList`、Python 的 `PyListObject`、C++ STL 的 `vector` 等实现。

倍增技术以 $O(N)$ 空间保存大小为 N 的可变长数组。它在很多应用中足够好，但即便只考虑增长操作，也可能在某些时刻浪费所分配内存的 50%。更一般地，采用增长因子 $1+\alpha$ 时，最多可能浪费约 $\alpha/(1+\alpha)$ 的内存；减小 α 能降低浪费比例，但增长操作的摊还代价会上升到 $O((1+\alpha)/\alpha)$ 。

能否仅用 $N+o(N)$ 空间保存大小为 N 的可变长数组，同时保持每次操作 $O(1)$ ？Sitariski [15] 和 Brodnik 等人 [2] 给出了肯定答案：他们的简单实现只用 $N+O(\sqrt{N})$ 空间。Brodnik 等人还证明，在一定意义下这是最优的：任何可变长数组实现都会在某些时刻使用 $N+\Omega(\sqrt{N})$ 空间。

本文提出，应区分两种空间：一是当前大小为 N 的数组在支持 $O(1)$ 最坏情形读写时所需的常驻空间；二是在 $O(1)$ 摊还时间内增减数组大小时短暂需要的空间。很多场景更重视紧凑的常驻表示，即便调整大小时需要更多临时空间。例如，一个应用可能同时使用很多可变长数组，而任一时只调整其中一个。

第一个突破 Brodnik 等人下界的实现，用 $N+O(N^{1/3})$ 空间存储数组，在调整大小时短暂使用 $N+O(N^{2/3})$ 空间。元素访问最坏情形为 $O(1)$ ，增减大小摊还为 $O(1)$ 。而且这一结果本质上最优：任何只用 $N+O(N^{1/3})$ 常驻空间的实现，都必须在某些调整时刻使用 $N+\Omega(N^{2/3})$ 空间。

更一般地，对每个 $r \geq 2$ ，可以用 $N+O(N^{1/r})$ 空间存储可变长数组，并在调整大小时使用 $N+O(N^{1-1/r})$ 空间。按下标读写为 $O(1)$ 最坏情形时间，调整大小为 $O(r)$ 摊还时间。除 $r=2$ 外，由于无法使用后台重建，这个 $O(r)$ 摊还界不能改成最坏情形界； $r=2$ 时的实现与 Sitariski 和 Brodnik 等人的结构非常相似。

文章结构如下：第 2 节形式化问题与计算模型；第 3 节回顾已有工作；第 4 节给出 Brodnik 等人的下界及其推广；第 5 节给出 $r=3$ 的简单方案；第 6 节给出一般 $r \geq 2$ 的实现；第 7 节通过变换把常驻额外空间从 $O(rN^{1/r})$ 降到 $O(N^{1/r})$ ；第 8 节定义并精确分析增长游戏；第 9 节据此证明增长操作 $\Omega(r)$ 的摊还下界；第 10 节给出结论。

2 可变长数组

可变长数组是一种支持下列操作的抽象数据类型：

- $A \leftarrow \text{Array}()$ 创建并返回一个初始为空的数组。
- $A.\text{Length}()$ 返回数组 A 的当前长度。

A.Get(i)	返回 A 中第 i 个元素 a_i , 假定 $0 \leq i < A.Length()$ 。
A.Set(i,a)	将 A 中第 i 个元素改为 a, 假定 $0 \leq i < A.Length()$ 。
A.Grow(a)	将 A 的长度增加 1, 并把新的末尾元素设为 a。
A.Shrink()	将 A 的长度减少 1, 丢弃最后一个元素。

元素只能在数组末端加入或删除。若允许在任意位置插入、删除并相应更新较高下标元素的下标, 问题会困难得多; 即便不要求空间效率, 所有操作能同时达到的最佳时间界也是 $O(\log N / \log \log N)$, 并且该界是紧的。对任意 $r \geq 1$, 可以让访问操作为 $O(r)$ 最坏情形时间、插入删除为 $O(N^{1/r})$ 摊还时间。本文始终只允许在末端增删。

我们假设内存管理系统能够分配和释放任意大小的定长数组, 例如 C/C++ 的 malloc 和 free。为简化叙述, 假定每次调用只需常数时间; 即使分配或释放大小为 N 的定长数组需要 $O(N)$ 时间, 本文的所有摊还界仍成立。我们不讨论内存管理器内部实现。

可变长数组实现必须使用一组动态分配的定长数组, 本文称为“块”。典型地, 块分为数据块、索引块和辅助块: 数据块逐字存放数组元素; 索引块逐字保存指向其他块的指针; 辅助块保存各块长度等信息。本文所有结构都属于此类。若当前数组大小为 N, 所有数据块的总大小至少为 N。除一个称为主索引块的索引块外, 其余每个块都必须由某个索引块中的指针指向, 否则无法访问。数据结构使用的空间定义为所有已分配块长度之和。

定义 2.1 (($s(N), t(N)$)-实现)。设 $s(N)$ 、 $t(N)$ 为两个非递减函数。若一种可变长数组数据结构在存储大小为 N 的数组时最多使用 $N+s(N)$ 空间, 在对大小为 N 的数组执行增长或收缩期间最多使用 $N+t(N)$ 空间, 则称它为一个 ($s(N), t(N)$)-实现。

3 以往工作

设计空间高效的可变长数组属于简洁数据结构领域。真正直接处理本文所定义基本问题的论文并不多, 下面介绍其中最相关的结果。

3.1 基本数据结构

基本实现只用一个定长数组来表示可变长数组。定长数组满时, 分配更大的数组, 将旧数组全部复制进去, 加入新元素并释放旧数组; 若若干次收缩后数组过空, 也可以分配更小的数组并复制。

最省常驻空间的基本实现让定长数组大小始终等于当前元素数。每次增长或收缩都要重新分配。存储大小为 N 的数组只需 $N+O(1)$ 空间, 但增长、收缩的摊还代价为 $\Omega(N)$, 且调整期间要同时保存新旧两个数组, 临时需要 $2N+O(1)$ 空间。它只适合调整很少发生的场景。

更实用的基本实现采用几何扩张/收缩。固定 $\alpha > 0$; 满时分配大小 $(1+\alpha)N$ 的新数组; 当容量为 N 的数组只含 $N/(1+\alpha)^2$ 个元素时, 缩到 $N/(1+\alpha)$ 。简单计算得到增长摊还代价为 $(1+\alpha)/\alpha$, 收缩摊还代价为 $1/\alpha$ 。恰当选择 α 通常能取得良好的工程折中。

3.2 Sitarski 的数据结构

Sitarski [15] 提出了称为哈希数组树 (HAT) 的简洁实现。名称略显误导：结构没有使用哈希，所谓“树”实际上是一条列表。

HAT 维护一个大小为 B 的索引块 I ，其中 $\sqrt{N} \leq B < 4\sqrt{N}$ ，并维护 $\lceil N/B \rceil$ 或 $\lceil N/B \rceil + 1$ 个大小为 B 的数据块。 N 个元素按顺序存入前 $\lceil N/B \rceil$ 个数据块；若 N 不能被 B 整除，最后一个仅部分填满；额外的下一个数据块若存在则为空。索引块第 i 项保存指向第 i 个数据块的指针。

该结构只使用 $N + 3B + 2 = N + O(\sqrt{N})$ 个内存字。第 i 个元素位于第 $\lceil i/B \rceil$ 个数据块的第 $i \bmod B$ 个位置，因此访问和修改均为 $O(1)$ 最坏情形时间。若 B 取 2 的幂，索引计算尤其简单。

当 $N < B^2$ 时，增长很直接：若最后数据块未滿，写入其末尾；若最后块已滿而已有一个空块，则写入空块首位；否则新分配一个大小为 B 的块。若分配视为 $O(1)$ ，增长最坏情形为 $O(1)$ ；若分配大小 B 的块需 $O(B)$ ，则增长摊还为 $O(1)$ 。

当结构滿，即 $N = B^2$ 时，把 B 加倍并整体重建。若初始 B 是 2 的幂，此性质一直保持。重建代价 $O(N)$ ，可分摊到自上次重建以来至少约 $3N/4$ 次增长上，因此增长摊还仍为 $O(1)$ 。

收缩类似。为避免数据块释放后立刻重分配，只有当最后两个数据块都为空时才释放最后一个。当 $B = 4\sqrt{N}$ （等价于 $N = B^2/16$ ）时，将 B 减半并重建。两次重建之间至少发生 $\Omega(N)$ 次操作，因此增长和收缩的摊还代价保持 $O(1)$ 。重建只需 $O(\sqrt{N})$ 额外空间，因为新旧数据块可以逐个分配与释放。

HAT 可以通过同时使用两种大小的数据块并在后台重建来去摊还化，但结构会更复杂。

图 1：Sitarski 的 HAT 数据结构。

3.3 Brodник 等人的数据结构

Brodnik 等人 [2] 在似乎不知道 HAT 的情况下，给出了一个优雅结构：只用 $N + O(\sqrt{N})$ 空间，并让增长和收缩达到 $O(1)$ 最坏情形时间（假设内存分配为 $O(1)$ 最坏情形）。它无需整体重建，数据元素也从不移動。

第一种版本把元素连续存入大小为 $1, 2, 3, \dots$ 的数据块。由于 $1 + 2 + \dots + k = k(k+1)/2 \approx k^2/2$ ，保存 N 个元素需要约 $\sqrt{2N}$ 个块，精确地 $k = \lceil (\sqrt{8N+1} - 1)/2 \rceil$ 。最后一个块可能部分填满；还可能保留一个完全空的第 $k+1$ 块。一个大小 $\Theta(\sqrt{N})$ 的索引块保存指针，该索引块本身用朴素可变量数组实现。

增长时，把新元素写入部分填满块的末尾，或写入空块首位；都不存在时，分配新数据块并在索引块中追加指针。索引块滿时分配更大的索引块并复制指针；要达到 $O(1)$ 最坏情形，这一复制需在后台进行。收缩类似。

访问第 i 个元素虽然为 $O(1)$ 最坏情形，但需要计算平方根：它位于第 $\ell = \lceil (\sqrt{8i+1} - 1)/2 \rceil$ 个数据块中的第 $i - \ell(\ell-1)/2$ 个位置。

为避免平方根，第二种版本让每个数据块大小为 2 的幂。对 $k = 0, 1, \dots$ ，定义由 $2^{\lceil k/2 \rceil}$ 个、每个大小为 $2^{\lfloor k/2 \rfloor}$ 的块组成的虚拟超级块。这样可以用移位操作求出元素所在超级块及其位置。

图 2：(a) Brodник 等人的基本版本；(b) 块大小均为 2 的幂的版本。

4 Brodник 等人的下界及其推广

定理 4.1. 任何维护可变量数组的数据结构，都必然在某些时刻使用 $N + \Omega(\sqrt{N})$ 空间，其中 N 是当前数组长度；即使只执行增长和访问操作也成立。

证明。执行 N 次增长后，设数据结构使用 k 个连续内存块。这些块必须容纳全部 N 个元素，总大小至少为 N ；每个块还必须有一个指针。若 $k \geq \sqrt{N}$ ，空间至少为 $N + k \geq N + \sqrt{N}$ 。若 $k < \sqrt{N}$ ，则最大块大小 $\ell > \sqrt{N}$ 。该块在数组只有 $N' \leq N$ 个元素时被分配；刚分配后，它还没有保存元素，已有 N' 个元素必须位于其他块中。因此当时总空间至少为 $N' + \ell > N' + \sqrt{N'}$ 。证毕。

该下界假定保存 N 个任意 w 位元素需要 N 个内存字，保存 k 个指针需要 k 个内存字。若总内存只有 2^{cw} ($0 < c < 1$) 个位置，则指针只需 cw 位， k 个指针约占 ck 个字；这只会把额外空间下界改变一个常数因子。

定理 4.2. 任意 $(s(N), t(N))$ -实现必须满足 $s(N)t(N) \geq N$ ；即使只支持增长和访问操作也成立。

证明。 N 次增长后，全部元素分布在若干连续内存块中。每个块需要一个指针，而常驻额外空间最多为 $s(N)$ ，所以块数最多为 $s(N)$ 。因此至少一个块 B 的大小不小于 $N/s(N)$ 。设分配 B 的增长操作编号为 $N' \leq N$ 。刚分配时 B 为空，因此当时临时额外空间至少为 $N/s(N)$ 。由 t 的单调性， $t(N) \geq t(N') \geq N/s(N)$ ，即得结论。

推论 4.3. 对任意整数 $r \geq 1$ ，若某数据结构只用 $N + O(N^{1/r})$ 空间存储大小为 N 的可变长数组，则它在某些增长操作期间必须使用 $N + \Omega(N^{1-1/r})$ 空间；即使只允许增长和访问也成立。

5 简单的 $(O(N^{1/3}), O(N^{2/3}))$ -实现

本节给出简单实现：访问最坏情形 $O(1)$ ，增长和收缩摊还 $O(1)$ ，常驻额外空间 $O(N^{1/3})$ ，调整期间额外空间 $O(N^{2/3})$ 。

基本思想是先考虑数据块大小约为 $N^{2/3}$ 的 HAT。索引块长度约为 $N^{1/3}$ ，但最后一个数据块可能几乎为空，从而总空间为 $N + O(N^{2/3})$ 。修正方法是再用一个 HAT 来管理这个大小为 $N^{2/3}$ 的部分填满块。于是常驻额外空间降为 $O((N^{2/3})^{1/2}) = O(N^{1/3})$ 。当末尾结构积累到 $N^{2/3}$ 个元素时，将它复制到一个新分配的大块中；复制期间使用 $\Omega(N^{2/3})$ 额外空间。

也可以基于 Brodник 等人的结构得到类似方案：依次使用大小 $1, 4, \dots, i^2, \dots, k^2$ 的块，随后再使用 $1, 2, 3, \dots$ 的小块。

非递归实现维护参数 B ，使 $N^{1/3} \leq B \leq 4N^{1/3}$ 。它使用约 N/B^2 个大小 B^2 的“大块”，以及至多 $2B$ 个大小 B 的“小块”；分别由两个索引块管理。大块始终填满。至多一个小块部分填满，至多一个小块为空。若 $2B$ 个小块全满，将其中 B 个复制到新分配的大块；若没有非空小块，则把一个大块拆成 B 个新分配的小块。所有访问、修改操作为 $O(1)$ 最坏情形；增长、收缩为 $O(1)$ 摊还时间。

图 3：一个简单的 $(O(N^{1/3}), O(N^{2/3}))$ -实现。

6 对任意 $r \geq 2$ 的 $(O(rN^{1/r}), O(N^{1-1/r}))$ -实现

推广第 5 节的构造, 对任意 $r \geq 2$ 可得一个 $(O(rN^{1/r}), O(N^{1-1/r}))$ -实现。结构维护参数 B , 使 $N^{1/r} \leq B < 4N^{1/r}$, 并把元素保存在大小为 $B, B^2, \dots, B^{r-1}$ 的块中。

若只支持增长, 可以维持每种大小的块不超过 B 个。为高效支持收缩, 后面把上限放宽到 $2B$ 。大小 $B^2, \dots, B^{r-1}$ 的块始终填满; 大小 B 的块除最后一个外也填满。若最后一个 B 块有空位, 增长直接写入; 否则若 B 块少于 B 个, 就分配新 B 块; 若某一级已有 B 个满的 B^i 块, 则分配一个 B^{i+1} 块, 把这 B 个块按顺序复制进去并释放旧块。这与 B 进制计数器的进位过程相同。

当 $N = B^r$ 时, 将 B 加倍并整体重建。也可以等结构完全填满到 $B^r + B^{r-1} + \dots + 1$ 后再重建, 但收益不大。

结构维护 $r-1$ 个索引块, 每种数据块大小一个。存储 N 个元素时, 总空间至多 $N + (2r-1) \cdot 2B + B = N + O(rN^{1/r})$ 。最大临时额外空间为 $O(N^{1-1/r})$, 发生在分配大小 B^{r-1} 的新块时。

直接实现时, 访问每个元素可在 $O(r)$ 最坏情形完成; 第 6.3 节把它降为 $O(1)$ 。增长摊还为 $O(r)$, 因为元素先进入 B 块, 之后至多逐级移动到 $B^2, \dots, B^{r-1}$ 。两次整体重建之间元素数增加常数倍, 因此重建只给每次增长增加常数摊还代价。

支持收缩时使用冗余 B 进制计数器: 每级允许最多 $2B$ 个块。若某级有 $2B$ 个满块, 则把前 B 个合并成上一级新块; 若收缩时没有 B 块, 则选择最小的非空 B^i 块, 把它拆成 $B-1$ 个各级 $B^{i-1}, \dots, B^2$ 块, 以及 B 个 B 块。增长和收缩的摊还代价仍为 $O(r)$ 。

图 4: 对任意 $r \geq 2$ 的 $(O(rN^{1/r}), O(N^{1-1/r}))$ -实现。

6.1 实现细节

记 n_i 为大小 B^i 的数据块数量, 块记作 $A[i][0], \dots, A[i][n_i-1]$ 。若 $n_1 > 0$, n_0 表示最后一个 B 块中已有的元素数。 $A[i]$ 是第 i 个索引块, 大小 $2B$; A 是指向这些索引块的指针块, 大小 r 。 $n_0, \dots, n_{r-1}$ 存在大小 r 的辅助块 n 中。

```
Grow(a):
  if  $N = B^r$ :
    Rebuild( $2B$ )
  else if  $n_1 = 2B$  and  $n_0 = B$ :
    Combine-Blocks()
  else if  $n_1 = 0$  or  $n_0 = B$ :
     $A[1][n_1] \leftarrow \text{Allocate}(B)$ 
     $n_1 \leftarrow n_1 + 1$ 
     $n_0 \leftarrow 0$ 
   $A[1][n_1-1][n_0] \leftarrow a$ 
   $n_0 \leftarrow n_0 + 1$ 
   $N \leftarrow N + 1$ 

Combine-Blocks():
   $k \leftarrow \min \{ i \text{ in } [r-1] \mid n_i < 2B \}$ 
```

```

if k = infinity: error
for i <- k-1 downto 1:
  A[i+1][n_{i+1}] <- Allocate(B^{i+1})
  for j <- 0 to B-1:
    Copy(A[i][j], 0, A[i+1][n_{i+1}], j B^i, B^i)
    Deallocate(A[i][j])
    A[i][j] <- A[i][j+B]
  ni <- B
  n_{i+1} <- n_{i+1} + 1

Shrink():
if N = (B/4)^r:
  Rebuild(B/2)
else if n1 = 0:
  Split-Blocks()
n0 <- n0 - 1
N <- N - 1
if n0 = 0:
  Deallocate(A[1][n1-1])
  n0 <- B
  n1 <- n1 - 1

Split-Blocks():
k <- min { i in [r-1] | ni > 0 }
if k = infinity: error
for i <- k-1 downto 1:
  n_{i+1} <- n_{i+1} - 1
  for j <- 0 to B-1:
    A[i][j] <- Allocate(B^i)
    Copy(A[i+1][n_{i+1}], j B^i, A[i][j], 0, B^i)
  Deallocate(A[i+1][n_{i+1}])

```

Allocate(ℓ) 分配长度 ℓ 的新块；Deallocate(X) 释放块 X ；Copy(X, x, Y, y, ℓ) 把 $X[x..x+\ell-1]$ 复制到 $Y[y..y+\ell-1]$ 。Rebuild(B') 用新参数 B' 重建结构。

Grow(a) 在 $N=B^r$ 时重建；第一级完全满时调用 Combine-Blocks。该过程找到最小 k ，使 $n_k < 2B$ ，然后从 $k-1$ 级向下逐级把前 B 个 B^i 块合并成一个 B^{i+1} 块。为保持顺序，剩余的后 B 个块只通过指针移动成为新的前 B 个块。

Shrink 在 $N=(B/4)^r$ 时把 B 减半并重建；若没有 B 块，则 Split-Blocks 找到最小的非空级别，把最后一个 B^k 块拆到各个较小级别。最后删除末尾元素；若一个 B 块变空就释放它。实际实现可延迟释放，以避免马上重新分配。

定理 6.1。上述实现是一个 $(O(rN^{1/r}), O(N^{1-1/r}))$ -可变长数组实现。增长和收缩的摊还代价为 $O(r)$ ，访问操作可支持 $O(1)$ 最坏情形时间。

推论 6.2。令 $r = \log N$ ，可得到一种实现：存储大小为 N 的数组只需 $N + O(\log N)$ 空间；增长和收缩为 $O(\log N)$ 摊还时间；访问为 $O(1)$ 最坏情形时间。

6.2 增长与收缩的摊还代价

引理 6.3。上述实现中，增长和收缩操作的摊还代价为 $O(r)$ 。

证明采用信用法。把一次增长或收缩的实际代价定义为它执行的元素赋值次数。若 Combine-Blocks 生成大小 B^k 的块，代价为 $B(B + B^2 + \dots + B^{k-1}) \leq 2B^k$ ($B \geq 2$)；若 Split-Blocks 拆分 B^k 块，代价为 B^k 。

当前位于第 i 级（大小 B^i 的块）的每个元素携带 $r-i$ 个信用单位。新元素总在第 1 级，因此增长本身的实际代价 1 加上 $r-1$ 个新信用，摊还代价为 r 。合并时，每次赋值把元素从第 i 级移到第 $i+1$ 级，元素信用减少 1，正好支付该次赋值，所以 Combine-Blocks 的摊还代价为 0。

为处理收缩，每次 Shrink 给每一级增加 3 个信用单位，因此不含 Split-Blocks 的摊还代价至多为 $3r$ 。两次在第 k 级拆分 B^k 块之间，至少发生 B^k 次收缩；因此该级已积累至少 $3B^k$ 信用。拆分的实际代价为 B^k ，给被移到低级别的元素补充信用最多 $2B^k$ ($B \geq 4$)，总计可由 $3B^k$ 支付。

最后处理重建。让每次增长或收缩再向整个结构加入 $2r$ 个信用。两次把 B 加倍的重建之间至少有 $N/2$ 次增长；两次把 B 减半的重建之间至少有 N 次收缩。重建前至少积累 rN 信用；实际重建代价恰为 N ，重新给全部元素分配信用最多需 $(r-1)N$ ，所以重建摊还代价为 0。由此总摊还代价为 $O(r)$ 。

6.3 $O(1)$ 最坏情形访问与修改

为简化访问，让 n_1 表示满的 B 块数量，部分填满的 B 块单独用 $n_0 < B$ 表示。假定 $B = 2^b$ 。于是 $N = \sum_{j=0}^{r-1} n_j B^j$ ，而 $(n_{r-1}, \dots, n_1, n_0)$ 是一个可能冗余的 B 进制表示。把每个 n_j 拆成 $n_j = n_j^0 + B n_j^1$ ，其中 $0 \leq n_j^0 < B$ 、 $0 \leq n_j^1 \leq 2$ ；将所有 n_j^0 和 n_j^1 分别紧凑拼接到两个机器字 N_0 、 N_1 中。

定义 $N_k = \sum_{j=0}^{k-1} n_j B^j$ 。 $N - N_k$ 是存于大小至少 B^k 的块中的元素数。第 i 个元素所在的唯一层级 k 满足 $N - N_{k+1} \leq i < N - N_k$ 。确定 k 后，块号与块内偏移都可通过移位和按位与计算。

令 $x = (N-1) - i$ ，即从数组末端起的下标。需要找唯一 k 使 $N_k \leq x < N_{k+1}$ 。先用最高置位 $\text{msb}(x)$ 求 x 的最高非零 B 进制位 $\ell = \lfloor \text{msb}(x)/b \rfloor$ 。可证明 k 至少为 $\ell-1$ ，随后只需比较 N_ℓ 、 $N_{\ell+1}$ ；若仍需向上找，就在 $N_0 \vee N_1$ 的高位部分求最低置位 lsb ，得到下一非空层级。由于 $\text{lsb}(y) = \text{msb}(y \wedge (-y))$ ，只需常数次位运算与 msb 。

在字 RAM 模型中， msb 可借助浮点指数、乘法技巧等在 $O(1)$ 时间求出。若不追求与 r 无关的理论常数，实践中用 $O(\log r)$ 二分甚至 $O(r)$ 顺序扫描通常更合适。

7 一种数据结构变换

当 r 为常数时，第 6 节的实现已经在临时空间和 $O(r) = O(1)$ 摊还代价方面最优。若 $r = r(N)$ 缓慢增长，一个自然问题是：能否把常驻额外空间 $O(rN^{1/r})$ 降到 $O(N^{1/r})$ ？增长、收缩的摊还代价是否必须为 $\Omega(r)$ ？只需考虑 $r = O(\log N)$ ，因为 $N^{1/O(\log N)} = O(1)$ 。

下面给出一个简单变换，在 $r \leq (1/2)(\log N / \log \log N)$ 时把常驻额外空间降为 $O(N^{1/r})$ 。

定义 7.1 (标准 $(s(N), t(N))$ -实现)。若一个实现满足：(i) 分配新块 B 时，立即把某些旧块 $A_1, \dots, A_k$ 的元素按顺序复制进去，并依次释放旧块；(ii) 每次这样的操作完成后，总空间回到 $N + s(N)$ ，则称其为标准实现。本文所有实现都是标准的。

引理 7.2. 任意标准 $(s(N), O(N))$ -实现，对任意非递减函数 $t(N)$ ，都可转化为 $(s(N) + O(N/t(N)), s(N) + O(t(N)))$ -实现，同时不改变访问操作的渐近最坏情形代价，也不改变增长、收缩的渐近摊还代价。

证明。若待分配新块 B 的长度 $b \leq t(N)$ ，照常分配；若 $b > t(N)$ ，就改为顺序分配并填充 $\lceil b/t(N) \rceil$ 个大小 $t(N)$ 的块。需要额外保存这些块的指针。由于任一时刻所有块总大小至多 $N + s(N)$ ，额外指针数最多为 $(N + s(N))/t(N)$ 。因此常驻空间至多 $N + s(N) + O(N/t(N))$ ，任一时刻总空间至多再加一个 $t(N)$ 大小的新块。假定 $t(N) \geq \sqrt{N}$ ，即得结论。

定理 7.3. 对任意 $r = r(N) \leq (1/2)(\log N / \log \log N)$ ，存在一个 $(O(N^{1/r}), O(N^{1-1/r}))$ -实现，访问和修改为 $O(1)$ 最坏情形时间，增长和收缩为 $O(r)$ 摊还时间。

证明。把引理 7.2 以 $t(N) = N^{1-1/r}$ 应用于第 6 节参数为 $2r$ 的标准实现。得到常驻额外空间 $O(rN^{1/(2r)} + N^{1/r})$ ，临时额外空间 $O(N^{1-1/r})$ 。当 $r \leq (1/2)(\log N / \log \log N)$ 时， $rN^{1/(2r)} \leq N^{1/r}$ ，故满足所需界。

8 增长游戏

为理解真正最优的数据结构应如何执行增长，并证明当标准数据结构只允许 $N + O(N^{1/r})$ 常驻空间时增长的摊还代价为 $\Omega(r)$ ，本文定义并精确分析一个单人游戏——增长游戏。

8.1 (N, k, ℓ) -增长游戏

固定 N, k, ℓ 。需要把 N 个元素逐个插入 k 个初始为空的子数组 $A_1, \dots, A_k$ ，它们共同表示可变长子数组 A 。A 的最前面元素存入 A_k ，之后依次向 A_{k-1} 等子数组排列。记 a_i 为 A_i 中元素数。

空子数组必须位于最前面：若 $a_i = 0$ ，则 $a_1 = \dots = a_i = 0$ 。第一个非空子数组可有至多 ℓ 个空位；其余非空子数组均满。因此常驻额外空间至多 $k + \ell$ 。

增长规则如下：(1) 第一个非空子数组有空位时，把新元素写入其第一个空位，代价 1；(2) 所有非空子数组都满但仍有空子数组时，分配大小 $\ell + 1$ 的新子数组，把新元素放入首位，留下 ℓ 个空位，代价 1；(3) 所有 k 个子数组都满时，可任选 $i \in [k]$ ，分配大小 $(\ell + 1) + \sum_{j=1}^i a_j$ 的新子数组，按 $A_i, A_{i-1}, \dots, A_1$ 的次序复制元素，再把新元素放入第一个空位。释放旧 $A_1, \dots, A_i$ ，新子数组成为 $A_i, A_1, \dots, A_{i-1}$ 变空。代价定义为 $1 + \sum_{j=1}^i a_j$ 。

定义 8.1. 令 $C_{\{N, k, \ell\}}$ 为完成 (N, k, ℓ) -增长游戏所需的最小总代价， $A_{\{N, k, \ell\}} = C_{\{N, k, \ell\}} / N$ 为单次增长的最小摊还代价。

图 6：增长游戏的三类移动：(a) 填补空位；(b) 新建一个带 ℓ 个空位的子数组；(c) 合并前 i 个子数组并加入新元素。

8.2 约化到 $\ell=0$

引理 8.2. 若 N 可被 $\ell+1$ 整除, 则 $C_{\{N,k,\ell\}}=(\ell+1)C_{\{N/(\ell+1),k,0\}}$, 且 $A_{\{N,k,\ell\}}=A_{\{N/(\ell+1),k,0\}}$ 。

证明。一次操作留下 ℓ 个空位后, 接下来的 ℓ 次增长只能依次填满它们。因此, (N,k,ℓ) -游戏等价于把每 $\ell+1$ 个元素视为一个整体, 在这些整体上进行 $(N/(\ell+1),k,0)$ -游戏。

8.3 $\ell=0$ 情形的分析

下文记 $C_{\{N,k\}}=C_{\{N,k,0\}}$ 、 $A_{\{N,k\}}=A_{\{N,k,0\}}$ 。状态集合 $P_{\{N,k\}}$ 由所有 $a=(a_1,\dots,a_k)$ 组成, 满足 a_i 为非负整数、 $\sum a_i=N$, 且空子数组只出现在前部。令 $C(a)$ 为从全零状态到达 a 的最小总代价, 则 $C_{\{N,k\}}=\min_{\{a \in P_{\{N,k\}}\}} C(a)$ 。

引理 8.4. 对任意 $a \in P_{\{N,k\}}$:

$$(i) C_k(a_1,\dots,a_k)=C_k(0,\dots,0,a_k)+C_{\{k-1\}}(a_1,\dots,a_{k-1});$$

$$(ii) C_k(a_1,\dots,a_k)=\sum_{j=1}^k C_j(0,\dots,0,a_j);$$

$$(iii) C_k(0,\dots,0,a_k)=C_{\{a_k-1,k\}}+a_k;$$

$$(iv) \text{ 若 } 0=a_{i-1}<a_i, \text{ 则 } C(a)=N+\sum_{j=i}^k C_{\{a_j-1,j\}}.$$

证明思路: 到达目标状态前, 最后一次改变 A_k 必须先到达 $(0,\dots,0,a_k)$, 之后只能使用前 $k-1$ 个子数组; 由此得到 (i), 递归得到 (ii)。到达 $(0,\dots,0,a_k)$ 的最后一次移动必从总大小 a_k-1 的某状态而来, 并花费 a_k , 由此得到 (iii) 与 (iv)。

引理 8.5 (组合恒等式)。对 $n,k \geq 0$:

$$(i) C(n,k)=C(n-1,k-1)+C(n-1,k);$$

$$(ii) \sum_{i=0}^k C(n+i-1,i)=C(n+k,k)。这里 C(n,k) 表示二项式系数。$$

定理 8.6。

$$(i) \text{ 若 } C(n+k-1,k)-1 \leq N \leq C(n+k,k)-1, \text{ 则 } C_{\{N,k\}}=(N+1)n-C(n+k,k+1)。$$

$$(ii) \text{ 若 } C(n+k-1,k) \leq N < C(n+k,k), \text{ 则 } C_{\{N,k\}}-C_{\{N-1,k\}}=n。$$

$$(iii) \text{ 若 } C(n+k-1,k)-1 \leq N < C(n+k,k)-1, \text{ 且 } a \in P_{\{N,k\}}, \text{ 则 } C(a)=C_{\{N,k\}} \text{ 当且仅当对每个 } i \in [k], \text{ 都有 } C(n+i-2,i) \leq a_i \leq C(n+i-1,i)。$$

定理说明: 当 $N=C(n+k,k)-1$ 时, 条件可同时由 n 与 $n+1$ 满足, 两种公式给出同一值。最优终态通常不唯一; 在边界 $N=C(n+k,k)-1$ 时, 唯一最优状态为 $(C(n,1), C(n+1,2), \dots, C(n+k-1,k))$ 。

定理 8.6 的证明采用对 (N,k) 的归纳。核心步骤包括: 利用 (iii) 的状态区间计算任一候选最优状态的精确代价; 若某坐标低于下界, 则必有另一个坐标高于相应下界; 若某坐标高于上界, 则必有另一坐标低于相应上界。把一个单位从“过大”的坐标移动到“过小”的坐标, 会依据增量公式严格降低总代价。因此, 且仅有区间内的状态最优。

推论 8.13。

$$(i) \text{ 若 } N=C(n+k,k)-1, \text{ 则 } C_{\{N,k\}}=[kn/(k+1)](N+1)。$$

$$(ii) \text{ 若 } C(n+k-1,k) \leq N \leq C(n+k,k)-1, \text{ 则 } C_{\{N,k\}} \geq [k/(k+1)](n-1)N, \text{ 因此 } A_{\{N,k\}} \geq [k/(k+1)](n-1) \geq (n-1)/2。$$

8.4 最优玩法：二项式计数器

借助定理 8.6 和引理 8.4 可以重构最优移动序列。通常最优序列不唯一；当且仅当 $N=C(n+k,k)-1$ 时唯一。

```
Initialize(k):
  a1,a2,...,ak <- 0
  b1,b2,...,bk <- 0
  b_{k+1} <- infinity

Increment(k):
  i <- min { j in [k] | b_j < b_{j+1} }
  a_i <- 1 + sum_{j=1}^i a_j
  b_i <- b_i + 1
  a1,...,a_{i-1} <- 0
  b1,...,b_{i-1} <- 0
```

引理 8.15。在任意时刻， $a_i=C(b_i+i-1,i)$ 。证明由移动次数归纳并应用“曲棍球杆”二项式恒等式。

引理 8.16。若 $b_1=\dots=b_k=n$ ，则 $N=\sum a_i=C(n+k-1,k)-1$ ，二项式计数器产生的移动序列正是该 (N,k) -增长游戏的唯一最优解。

图 7：二项式计数器伪代码。图 8： $k=4$ 时计数器的运行示例。

8.5 扩展规则

首先允许一种不加入新元素的合并移动：对 $1 \leq i < j \leq k$ ，将 $A_i, \dots, A_j$ 合并为新的 A_j ； $A_1, \dots, A_{i-1}$ 向后平移，其余更前位置变空。代价为 $\sum_{\ell=i}^j a_\ell$ 。

引理 8.17。加入上述移动不会改变 (N,k) -增长游戏的最优值。证明利用引理 8.4 展开状态代价，说明任何新移动都可以由原规则以不更高代价模拟。

进一步允许任意集合 $I=\{i_1 < \dots < i_r\}$ 的 I -移动：把这些非连续子数组及一个新元素合并为新的 $A_{\{i_r\}}$ ，其余被合并位置变空，代价为 $1+\sum a_{\{i_\ell\}}$ ；也可允许不加入新元素的版本。

引理 8.18。允许任意 I -移动仍不改变最优值。加入这些扩展后，增长游戏已经覆盖所有标准实现可能执行的移动。

9 增长操作摊还代价的下界

定理 9.1。任何标准可变量数组数据结构，只要用 $N+O(rN^{1/r})$ 空间存储大小为 N 的数组，且 $r=O(\log N)$ ，其增长操作的摊还代价必为 $\Omega(r)$ 。

证明。可把常驻额外空间写为 $rN^{1/r}$ ；若常数因子为 $\alpha > 1$ ，可对 $r-1$ 应用结论。标准实现的操作对应扩展的 $(N, rN^{1/r}, rN^{1/r})$ -增长游戏。由引理 8.2 和推论 8.13，其摊还代价至少为

$A_{\{N, rN^{1/r}, rN^{1/r}\}} = A_{\{N^{1-1/r}/r, rN^{1/r}\}} \geq n/2$ ，只要 $C(n+rN^{1/r}, n) < N^{1-1/r}/r$ 。

取 $n = \beta r$ 。利用二项式系数上界 $C(m, q) \leq (em/q)^q$ ，可得 $C(\beta r + rN^{1/r}, \beta r) \leq [e(\beta + 1)/\beta]^{\beta r} N^{\beta}$ 。取 $\beta = 1/2$ ，并令 $3 \leq r \leq \alpha \log N$ (α 充分小)，有 $r(3e)^{r/2} N^{1/2} \leq N^{3/4} \leq N^{1-1/r}$ 。因此 $n = \Theta(r)$ ，摊还代价为 $\Omega(r)$ 。

10 结束语

本文给出了一族可变长数组实现，在三者之间达到最优折中：存储数组的常驻空间、调整大小所需的临时空间、增长与收缩的摊还代价；同时访问仍保持 $O(1)$ 最坏情形时间。我们认为这个基础问题已经几乎完全解决。

关于增长摊还代价的下界目前只证明于“标准算法”。作者相信，在对元素与指针作适当不可压缩性假设后，下界可扩展到所有算法；换言之，非标准算法相较标准算法不会获得优势。

为得到摊还下界，本文定义并精确求解了增长游戏。其精确解本身具有启发性；仍值得探索是否存在更简单的归纳方法，直接得到足以证明渐近最优下界的估计。

参考文献

- [1] Philip Bille, Anders Roy Christiansen, Mikko Berggren Etienne, and Inge Li Gørtz. Fast dynamic arrays. ESA, 2017.
- [2] Andrej Brodnik, Svante Carlsson, Erik D. Demaine, J. Ian Munro, and Robert Sedgwick. Resizable arrays in optimal time and space. WADS, 1999.
- [3] Andrej Brodnik, Peter Bro Miltersen, and J. Ian Munro. Trans-dichotomous algorithms without multiplication. WADS, 1997.
- [4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to Algorithms, 3rd ed., MIT Press, 2009.
- [5] Paul F. Dietz. Optimal algorithms for list indexing and subset rank. WADS, 1989.
- [6] Michael L. Fredman and Michael E. Saks. The cell probe complexity of dynamic data structures. STOC, 1989.
- [7] Michael L. Fredman and Dan E. Willard. Surpassing the information theoretic bound with fusion trees. JCSS, 1993.
- [8] Michael T. Goodrich and John G. Kloss II. Tiered vectors: Efficient dynamic arrays for rank-based sequences. WADS, 1999.
- [9] Stelios Joannou and Rajeev Raman. An empirical evaluation of extendible arrays. SEA, 2011.
- [10] Haim Kaplan, Robert E. Tarjan, Or Zamir, and Uri Zwick. Simulating a stack using queues. SODA, 2022.
- [11] Jyrki Katajainen. Worst-case-efficient dynamic arrays in practice. SEA, 2016.

- [12] Donald E. Knuth. The Art of Computer Programming, Vol. 4, Fascicle 1. Addison-Wesley, 2009.
- [13] J. Ian Munro and S. Srinivasa Rao. Succinct representation of data structures. Handbook of Data Structures and Applications, 2018.
- [14] Rajeev Raman, Venkatesh Raman, and S. Srinivasa Rao. Succinct dynamic data structures. WADS, 2001.
- [15] Edward Sitarski. HATs: Hashed array trees. Dr. Dobbs' s Journal, 1996.
- [16] Wikipedia contributors. Dynamic array, 2022.
- [17] Wikipedia contributors. Hashed array tree, 2022.